

SSM-Adapter

Architecture Reference

Bidirectional Selective State-Space Temporal Adapter for Gloss-Free Sign Language Translation

Module: **SSM-Adapter (bidirectional selective state-space temporal adapter)**

Codebase: `src/pgat_length/models/ssm_adapter.py` (pgat-length) and
`external_baselines/TSPNet/fairseq/models/ssm_adapter.py` (TSPNet integration)

Author: Samir Pokharel (st125989)

Version: 2026-09-11

1 Purpose

SSM-Adapter is a small trainable neural-network module inserted between the visual encoder and the language decoder of a gloss-free sign language translation pipeline. Its job is to enrich each visual token in a sequence with long-range temporal context from all the other tokens in the same sequence, without changing the token count, the token dimension, or the training loss of the host pipeline.

The module is designed to be backbone-agnostic: the same module is used inside two independent systems (pgat-length with mBART, TSPNet with a fairseq transformer). It plugs in as a drop-in residual block at the visual-encoder output.

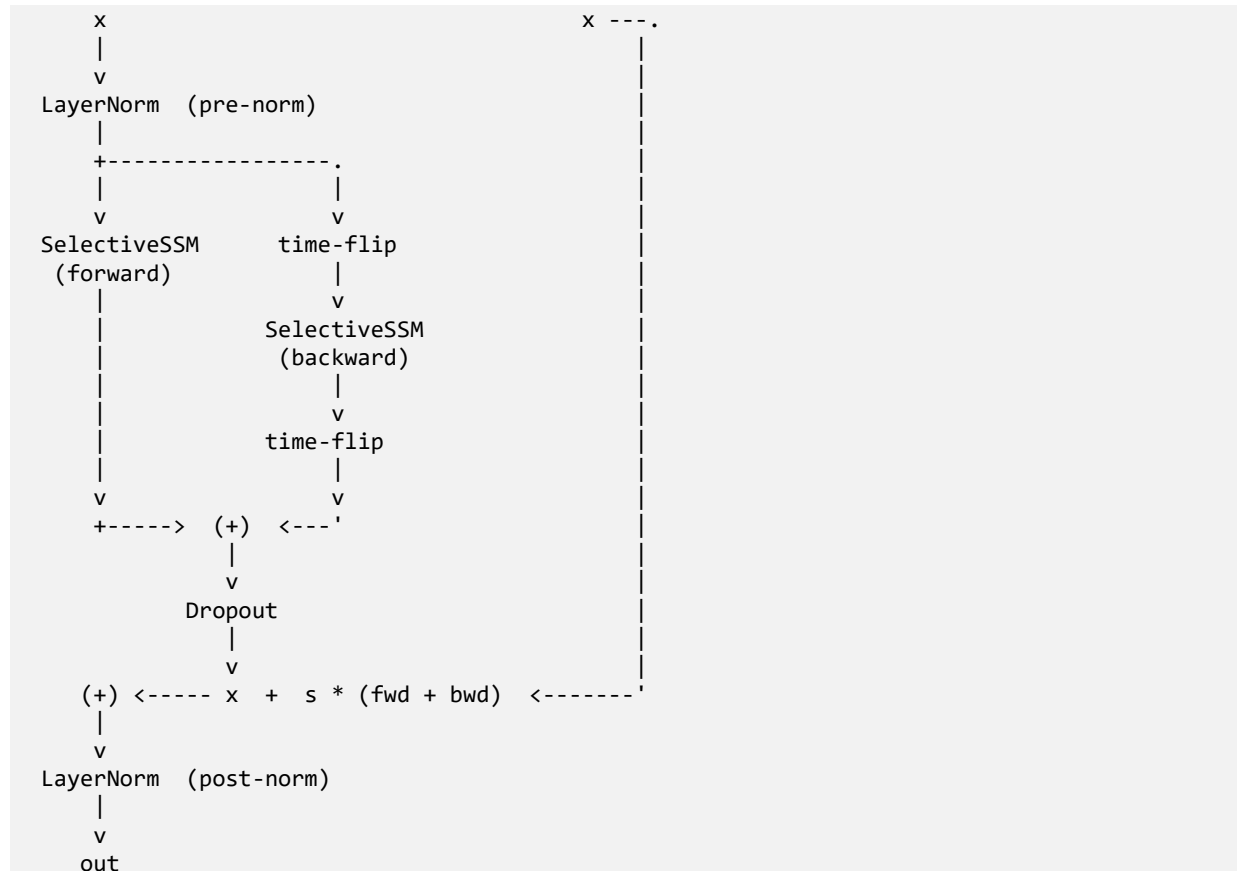
2 Notation

Symbol	Meaning	pgat-length	TSPNet
B	Batch size	varies	varies
T	Temporal sequence length	up to 32	a few hundred (multi-scale concat)
D	Model hidden dimension	512	1024
d_state	SSM state size per channel	16	16
d_conv	Depthwise conv kernel	4	4
expand	Inner-dim expansion factor	2	2
d_inner	Inner (expanded) dim = $\text{expand} \times D$	1024	2048
L	Layers per direction	1	1
s	Residual scaling factor	1.0	1.0

The input tensor to the adapter has shape $[B, T, D]$; the output tensor has the same shape.

3 Overall architecture

The adapter is a single residual block with the following top-level dataflow. All shapes are $[B, T, D]$.



The wrapper does five things in order:

- Apply a pre-normalisation (LayerNorm) to the input.
- Run the pre-normalised input through the selective SSM in the forward temporal direction.
- Run the pre-normalised input through the selective SSM in the backward temporal direction. The backward direction is implemented by time-reversing the input, running the same block, then time-reversing the output back to the original order.
- Sum the two directions, apply dropout, add the result as a residual to the original input (scaled by the residual scale s).
- Apply a post-normalisation (LayerNorm).

Because the module is a residual block, when the SSM parameters are randomly initialised at the start of training its contribution is small; the output is approximately equal to the input plus a small correction.

This guarantees the module cannot destroy the visual representation at initialisation — the model degrades gracefully to the backbone baseline if the SSM learns nothing.

Padded (invalid) positions in the input sequence contribute zero to the state update because their input values are already zero, and they are masked to zero in the output before the residual sum. This preserves the segment-validity mask through the adapter.

4 The bidirectional wrapper

Given an input tensor $x \in \mathbb{R}^{(B \times T \times D)}$ and an optional validity mask $m \in \{0,1\}^{(B \times T)}$ (where $m_{\{b,t\}} = 1$ marks a valid position), the wrapper computes:

$$\begin{aligned}\hat{x} &= \text{LayerNorm_pre}(x) \\ o_{\text{fwd}} &= \text{SSM_fwd}(\hat{x}, m) \\ o_{\text{bwd}} &= \text{flip_t}(\text{SSM_bwd}(\text{flip_t}(\hat{x}), \text{flip_t}(m))) \\ o &= \text{Dropout}(o_{\text{fwd}} + o_{\text{bwd}}) \\ \text{out} &= \text{LayerNorm_post}(x + s \cdot o) \odot m\end{aligned}$$

Here flip_t is a temporal reversal along the time axis, and $\odot m$ zeros out padded positions in the output.

The forward and backward SSM blocks have separate weights; they are not tied. The choice to sum rather than concatenate the two directions is a design decision that keeps the output dimensionality equal to the input dimensionality (so downstream code does not need to change).

5 The selective SSM block

Each direction contains a selective state-space block in the style of Mamba (Gu & Dao, 2023). The block operates on the pre-normalised input $\hat{x} \in \mathbb{R}^{(B \times T \times D)}$ and produces an output of the same shape. Its computation is composed of the following stages.

5.1 Input projection

An input linear layer $W_{\text{in}} \in \mathbb{R}^{(2 \cdot d_{\text{inner}} \times D)}$ expands the hidden dimension from D to $2 \cdot d_{\text{inner}}$ and splits the result into two streams: the SSM stream x_{ssm} and the gating stream z .

$$[x_{\text{ssm}}, z] = W_{\text{in}} \cdot \hat{x}, \quad x_{\text{ssm}}, z \in \mathbb{R}^{(B \times T \times d_{\text{inner}})}$$

Both streams have the same shape. The SSM stream is fed into the state-space computation; the gating stream is held aside and used at the end to modulate the output.

5.2 Depthwise convolution

A one-dimensional depthwise convolution $\text{DWConv}_{\{d_{\text{conv}}\}}$ operates along the time axis with kernel size $d_{\text{conv}} = 4$ and one filter per channel:

$$y_{\text{conv}} = \text{SiLU}(\text{DWConv}_{\{d_{\text{conv}}\}}(x_{\text{ssm}}))$$

The convolution is padded on the right to preserve the temporal length; excess positions are trimmed to T . The SiLU activation (also called Swish) is applied element-wise.

The purpose of this step is a short-range temporal mixing before the state-space part. It lets the model consume the last few timesteps in a lightweight way before committing to a hidden-state update.

5.3 Selective parameters

A linear layer W_{proj} reads the post-convolution tensor and produces three per-timestep quantities: the timestep scale $\Delta_t \in \mathbb{R}^{(d_{inner})}$, the input-projection matrix $B_t \in \mathbb{R}^{(d_{state})}$, and the output-projection matrix $C_t \in \mathbb{R}^{(d_{state})}$.

$$[\Delta, B, C] = W_{proj} \cdot y_{conv}, \quad W_{proj} \in \mathbb{R}^{((d_{inner} + 2 d_{state}) \times d_{inner})}$$

The scale is then passed through a softplus with a learned bias to ensure it is strictly positive:

$$\Delta_t = \text{softplus}(\Delta_t + \Delta_{bias})$$

The bias $\Delta_{bias} \in \mathbb{R}^{(d_{inner})}$ is initialised so that initial values of Δ are log-uniformly distributed in the interval $[10^{-3}, 10^{-1}]$. This spread is what makes the initial scan behave stably.

The fact that Δ , B , and C depend on the input y_{conv} at each timestep is what distinguishes Mamba from earlier state-space models like S4. In S4, these parameters are fixed for the entire sequence; in Mamba they are recomputed at each timestep. This selectivity lets the model use different dynamics for different positions — for example, remembering strongly at content-heavy positions and forgetting quickly at silent frames.

5.4 The state matrix A

The state-transition matrix $A \in \mathbb{R}^{(d_{inner} \times d_{state})}$ is a learned parameter, stored in log-space as $\log(-A)$ so that $A = -\exp(\log(-A))$ is always negative. Negative dynamics ensure the continuous-time system has decaying (stable) behaviour.

A is initialised using the S4D-lin scheme (Gu et al., 2022):

$$A_{\{i, n\}} = -(n + 1), \quad n = 0, 1, \dots, d_{state} - 1$$

for each channel i . This initialisation gives channel-independent, monotonically-spaced decay rates that are known to train stably.

5.5 Discretisation

The continuous-time state-space parameters are discretised to a per-timestep recurrence using the zero-order hold approximation (first-order for B):

$$\begin{aligned} \bar{A}_t &= \exp(\Delta_t \odot A), & \bar{A}_t &\in \mathbb{R}^{(B \times d_{inner} \times d_{state})} \\ \bar{B}_t &= \Delta_t \odot B_t, & \bar{B}_t &\in \mathbb{R}^{(B \times d_{inner} \times d_{state})} \end{aligned}$$

Here $\odot$ denotes broadcasted element-wise multiplication. $\bar{A}_t$ is the effective decay per state; $\bar{B}_t$ is the effective input weight per state.

5.6 Sequential scan

The core recurrence is a hidden-state update per channel per state, executed sequentially over the time axis. The hidden state $h_t \in \mathbb{R}^{(B \times d_{\text{inner}} \times d_{\text{state}})}$ starts at zero:

$$h_0 = 0$$

For $t = 1, 2, \dots, T$:

$$h_t = \bar{A}_t \odot h_{t-1} + \bar{B}_t \odot y_{\text{conv}, t} \\ o_t = \sum_n C_{\{t, n\}} \cdot h_{\{t, \cdot, n\}}, \quad o_t \in \mathbb{R}^{(B \times d_{\text{inner}})}$$

The recurrence is linear in the sequence length. Because T is small in our setting (up to 32 for pgat-length, up to a few hundred for TSPNet after multi-scale concatenation), the scan is implemented as a Python loop over the time axis using standard PyTorch primitives. No specialised CUDA kernel is required.

5.7 Skip connection and gating

The scan output o is combined with two auxiliary paths.

Skip. A per-channel learned scalar vector $D \in \mathbb{R}^{(d_{\text{inner}})}$ provides a direct shortcut from y_{conv} to the output:

$$o = o + D \odot y_{\text{conv}}$$

Gate. The gating stream z from Section 5.1 is passed through SiLU and multiplied element-wise with the output:

$$o = o \odot \text{SiLU}(z)$$

The gate acts as an on-off switch per timestep per channel. Together, the skip and the gate give the model two additional degrees of freedom on top of the state-space dynamics: the skip lets the model bypass the SSM when useful, and the gate lets it suppress features it does not want to expose.

5.8 Output projection

An output linear layer $W_{\text{out}} \in \mathbb{R}^{(D \times d_{\text{inner}})}$ maps the inner dimension back to the model hidden dimension:

$$o_{\text{block}} = W_{\text{out}} \cdot o, \quad o_{\text{block}} \in \mathbb{R}^{(B \times T \times D)}$$

This is the output of one direction. Padded positions are zeroed:

$$o_{\{block, b, t\}} \leftarrow o_{\{block, b, t\}} \odot m_{\{b, t\}}$$

5.9 Summary of the block

Putting the pieces of Section 5 together, one selective SSM block computes a function

$$\text{SSM}(\hat{x}, m) \rightarrow o_{\text{block}}, \quad o_{\text{block}} \in \mathbb{R}^{(B \times T \times D)}$$

with the intermediate steps: input projection $\rightarrow$ depthwise conv $\rightarrow$ selective parameter projection $\rightarrow$ softplus of Δ $\rightarrow$ discretisation of A and B $\rightarrow$ sequential scan $\rightarrow$ skip + gate $\rightarrow$ output projection.

6 Parameter budget

The adapter's total trainable parameter count is dominated by the four linear projections in each SSM block, which have quadratic-in-dimension parameter counts. For one direction:

Component	Parameter count
Input projection W_{in}	$2 \cdot d_{inner} \cdot D$
Depthwise conv	$d_{inner} \cdot d_{conv}$
Selective parameter projection W_{proj}	$(d_{inner} + 2 \cdot d_{state}) \cdot d_{inner}$
Softplus bias Δ_{bias}	d_{inner}
State matrix A (log-parameterised)	$d_{inner} \cdot d_{state}$
Skip vector D	d_{inner}
Output projection W_{out}	$d_{inner} \cdot D$

Instantiated for the two backbones:

Backbone	D	d_{inner}	Params per direction	Total (2 directions + 2 LayerNorms)
pgat-length	512	1024	~2.5 M	~5.36 M
TSPNet	1024	2048	~10.4 M	~21.2 M

Both values are small relative to the host backbones (25.7 M for the TSPNet baseline transformer, ~344 M for the pgat-length v2 mBART pipeline). The adapter can be trained end-to-end with the rest of the model or, in a lower-compute variant, with the backbone frozen.

7 Complexity

The forward pass has three cost regions.

- Projections and depthwise conv: linear in T, quadratic in the hidden dimensions. Amortised cost per timestep is $O(D \cdot d_{inner})$.
- Sequential scan: linear in T, with per-step cost $O(d_{inner} \cdot d_{state})$. The scan is executed as a Python loop; because $T \leq 32$ for pgat-length and T is a few hundred for TSPNet, the loop overhead is negligible relative to the projections.
- Total forward cost: $O(T \cdot d_{inner} \cdot (D + d_{state}))$.

For comparison, a transformer self-attention block over the same sequence has cost $O(T^2 \cdot D)$ — quadratic in T . At our sequence lengths this difference is small, but the linear-in- T scaling is what makes state-space models attractive for longer sequences that other applications might encounter.

Memory during the scan is dominated by the intermediate tensors $\bar{A}_t$ and $\bar{B}_t$, each of shape $[B, T, d_{\text{inner}}, d_{\text{state}}]$. At the TSPNet configuration ($B \approx 4$, $T \approx 300$, $d_{\text{inner}} = 2048$, $d_{\text{state}} = 16$), each intermediate is ≈ 1.5 GB in float32, or half that in bfloat16. To fit on a 24 GB GPU (e.g., RTX 4090) alongside the rest of the pipeline, the expansion factor can be reduced from 2 to 1, which halves d_{inner} and halves this memory.

8 Backbone integration

The adapter is inserted at the same position in both backbones: after the visual encoder produces its output token sequence and before the downstream heads consume it.

pgat-length. The adapter is placed between the PgatVariableTokenizer output and the articulator/global-summary heads. The input tensor has shape $[B, K, 512]$ where $K \in [12, 32]$ is the number of temporal segments for the sample. The validity mask is the segment-validity mask produced by the tokenizer. The adapter's output is then consumed by the articulator attention, the global-summary attention, the projection to mBART hidden dim, and the mBART encoder-decoder pipeline. All of these downstream components are unchanged.

TSPNet. The adapter is placed immediately after the three-scale I3D features are projected, positional-embedded, and concatenated along the sequence dimension, and immediately before the fairseq canonical $[T, B, C]$ transpose. The input tensor has shape $[B, T_{\text{concat}}, 1024]$ where T_{concat} is the sum of the three scales' sequence lengths. The validity mask is the concatenated per-scale padding mask. The adapter's output feeds into the standard fairseq transformer encoder-decoder pipeline. Everything downstream is unchanged.

In both cases the adapter is opt-in via a configuration flag. Without the flag, the model reduces to the corresponding backbone baseline byte-for-byte.

9 Configuration interface

The adapter is parameterised by a small configuration object:

Field	Type	Default	Meaning
d_model	int	512 or 1024	Model hidden dimension (must match the backbone).
d_state	int	16	SSM state dimension per channel.
d_conv	int	4	Depthwise conv kernel size.
expand	int	2	Inner-dim expansion factor.

Field	Type	Default	Meaning
num_layers	int	1	Number of stacked selective SSM blocks per direction.
dropout	float	0.1	Dropout probability applied to the summed forward+backward output before the residual add.
dt_min	float	0.001	Lower bound of the initial log-uniform range for the softplus bias on Δ .
dt_max	float	0.1	Upper bound of the same range.
residual_scale	float	1.0	Multiplicative scale on the SSM output before the residual add.

All of these are exposed as CLI flags in the TSPNet integration (`--use-ssm-adapter`, `--ssm-d-state`, `--ssm-d-conv`, `--ssm-expand`, `--ssm-num-layers`, `--ssm-dropout`, `--ssm-residual-scale`) and as YAML fields in the pgat-length integration (configs/model_ssm.yaml, under an `ssm_adapter`: key).

10 Code map

Symbol / concept	File	Symbol
Selective SSM block (one direction)	src/pgat_length/models/ssm_adapter.py	_SelectiveSSMBlock
Bidirectional wrapper	same file	BidirectionalSSMAdapter
Config dataclasses	same file	SSMAdapterConfig
pgat-length integration	src/pgat_length/models/translation.py	PgatMbartTranslationModel.encode_visual_prefix
TSPNet integration	external_baselines/TSPNet/fairseq/models/transformer_from_sign.py	TransformerEncoderSign.forward
pgat-length config	configs/model_ssm.yaml	ssm_adapter: (top-level key)

Symbol / concept	File	Symbol
TSPNet runner (SSM variant)	external_baselines/TSPNet/run_scripts/run_phoenix_pos_embed_sp_test_3lvl_ssm.sh	--use-ssm-adapter + --ssm-* flags

11 Design choices, briefly

Several design decisions in the adapter are documented here so that alternatives can be compared without re-deriving the reasoning.

Sum rather than concatenate the two directions. Summation keeps the output dimensionality the same as the input; concatenation would double it and require a projection to restore the shape. Since the two directions carry complementary information about the same tokens, summation is a reasonable low-parameter combiner.

Residual + LayerNorm outer structure. This matches the pre-norm transformer convention and guarantees that a randomly-initialised adapter is close to the identity at the start of training. Without the residual, the model would see garbage until the SSM learns.

S4D-lin initialisation of A. Alternative initialisations (HiPPO-LegS, uniform random) are unstable at our scale during the first few training steps. The S4D-lin scheme is empirically the most stable for a straight PyTorch implementation without the specialised state-space initialiser routines.

Pure PyTorch scan. The Mamba paper ships a CUDA kernel that fuses the scan for large sequences. We deliberately do not depend on it because (a) the kernel has a fragile build against specific PyTorch/CUDA combinations, and (b) our sequence lengths are small enough that the Python loop overhead is negligible.

Bidirectional rather than forward-only. Gloss-free translation is an offline task — the full video is available before generation. There is no reason to restrict the adapter to causal context, and using both directions lets each timestep condition on both what came before and what came after.

12 Limitations

The adapter is a small module and the study makes only bounded claims about it.

- The adapter increases parameter count and training time. On pgat-length it adds ~5 M parameters; on TSPNet it adds ~21 M. Wall time per epoch increases by roughly 10-20% due to the sequential scan.
- The adapter alone does not close the length cliff observed on PHOENIX-2014T. On pgat-length development, adding the adapter yields a small BLEU-4 improvement (7.36 → 7.61) and a

measurably flatter chrF curve (long-over-short ratio $0.772 \rightarrow 0.807$), but the absolute long-bin performance remains far below short-bin performance. The claim of this thesis is a small, measurable, cross-backbone-consistent flattening, not the elimination of the cliff.

- The Python scan is slower than the Mamba CUDA kernel by an estimated factor of 5-10x. For our short sequences this difference is not material.
- Only two backbones are used for the cross-backbone verification. A stronger claim of full backbone-agnostic behaviour would require additional systems and is left as future work.

References

- Gu, A., Goel, K., & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces (S4). ICLR.
- Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
- Li, K., Wang, Y., et al. (2024). VideoMamba: State Space Model for Efficient Video Understanding. ECCV.
- Camgoz, N. C., Hadfield, S., Koller, O., Ney, H., & Bowden, R. (2018). Neural Sign Language Translation. CVPR.
- Li, D., Xu, C., Yu, X., Zhang, K., Swift, B., Suominen, H., & Li, H. (2020). TSPNet: Hierarchical Feature Learning via Temporal Semantic Pyramid for Sign Language Translation. NeurIPS.
- Zhou, B., et al. (2023). Gloss-Free Sign Language Translation: Improving from Visual-Language Pretraining. ICCV.